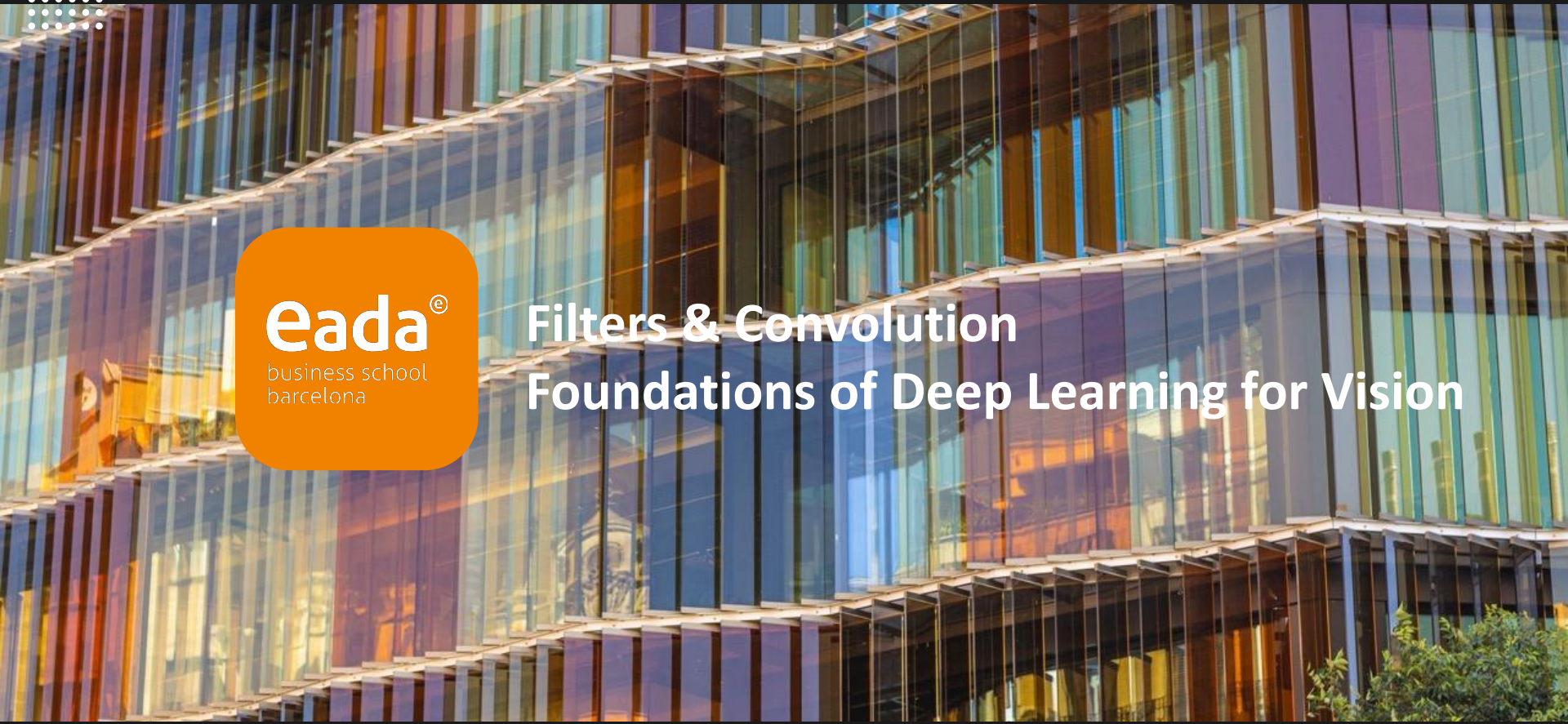


eada business school barcelona

Filters & Convolution
Foundations of Deep Learning for Vision

SESSION 4

Learning Objectives

- **Explain** what an image filter is, as a local operation applied at every pixel.
- **Compute**, by hand and in code, the convolution of a small image patch with a kernel.
- **Distinguish** blur, sharpen and edge-detection kernels, and predict what each will do to an image.
- **Describe** padding and stride, and how they control output size.
- **Connect** hand-designed kernels to the *learned* kernels inside a convolutional neural network.

SESSION 4

Today's Agenda

PART 1 — THEORY	BREAK	PART 2 — HANDS-ON PYTHON
80 min	20 min	80 min

- **Part 1:** filters, the convolution operation, padding/stride, and classical kernels (blur, sharpen, edges).
- **Break**: 20 minutes.
- **Part 2**: implement convolution from scratch in Python, then apply it to a real MILK10k image.

SESSION 4

Part 1 Outline — Theory

- What a filter is, and the sliding-window intuition.
- The convolution operation, defined formally and worked by hand.
- Padding and stride — controlling the size of the output.
- Classical kernels: blur (box, Gaussian), sharpen, and edge detection (Sobel).
- Kernels as feature detectors — and the bridge to learned convolution in deep learning.

SESSION 4

Part 1 — Theory

From pixel grids to feature detectors

eada
business school
barcelona

Recap — An Image Is a Matrix

- A digital image is a matrix (or 3 stacked matrices for RGB) of intensity values, typically `uint8` in [0, 255].
- So far, every operation we've applied has worked on **one pixel at a time** — normalization, per-channel scaling, grayscale conversion.
- Today's question: what can we compute using not just one pixel, but its **local neighborhood**?
- That single idea — combine a pixel with its neighbors, using fixed weights — is the entire foundation of both classical filtering and every convolutional neural network.

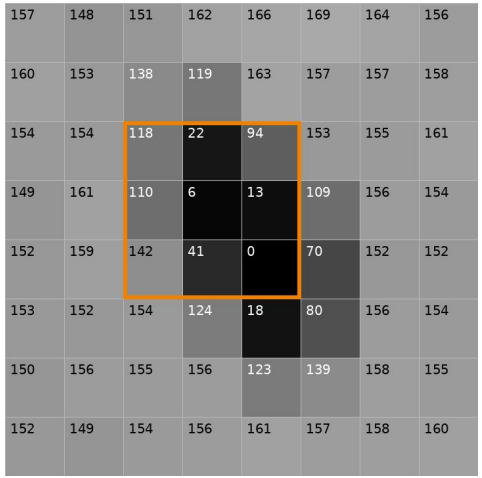
What Is a Filter?

? *You already know pixel-wise operations. What could you compute using a pixel's NEIGHBORS instead?*

- A **filter** (or **kernel**) is a small matrix of numbers — commonly 3×3 or 5×5.
- To apply it: slide the kernel over every position in the image; at each position, combine the kernel with the pixels underneath it to produce **one output value**.
- The collection of all these output values forms a new image, the **feature map** — same general idea as the input, but transformed to highlight something specific: edges, smoothness, texture.
- Which pattern gets highlighted depends entirely on the **numbers inside the kernel** — that is the one design choice that defines a filter.

The Sliding-Window Idea

- Each cell shows a real pixel intensity (0–255) from a MILK10k image, darker = lower value.
- The orange 3×3 window marks **one position** of the kernel as it slides left-to-right, top-to-bottom across the whole image.
- At every position, the 9 numbers under the window combine with the kernel's 9 weights to produce a single output pixel.



An 8×8 crop of real pixel values, with one 3×3 sliding-window position highlighted

The Convolution Operation — Formally

? *One formula underlies blur, sharpen, and edge detection. What could it possibly be?*

- For an image **I** and a kernel **K** of size (2a+1)×(2a+1), the output at position (x, y) is:
- **$(I * K)(x, y) = \Sigma_i \Sigma_j I(x+i, y+j) \cdot K(i, j)$**, for i, j from −a to a
- In words: **multiply** every kernel weight by the pixel underneath it, and **sum** the results — one multiply-accumulate per kernel cell, repeated at every image position.
- This is the single arithmetic operation underlying blur, sharpen, edge detection — and, every layer of a CNN.

KEY IDEA

Convolution replaces one pixel with a **weighted sum of its neighborhood**, the weights (the kernel) decide what gets emphasized.

Worked Example — Convolving by Hand

REAL PIXELS from the 8×8 crop

118	22	94
110	6	13
142	41	0

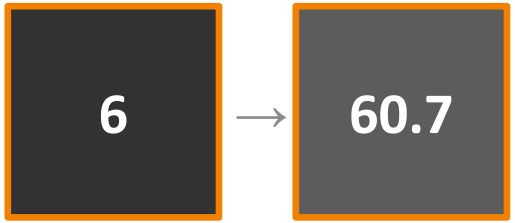
×

BOX-BLUR KERNEL × 1/9

1	1	1
1	1	1
1	1	1

→

CENTRE PIXEL before → after



Σ (118 + 22 + 94 + 110 + 6 + 13 + 142 + 41 + 0) / 9 = 546 / 9 = **60.7**

- Every weight is 1/9, so the output is just the average of the 9 pixels.
- The centre was 6 (almost black); its brighter neighbours pull it up to 60.7. That is blur: sharp jumps get smoothed.

Our Toy Image: 25 Numbers, Every Kernel

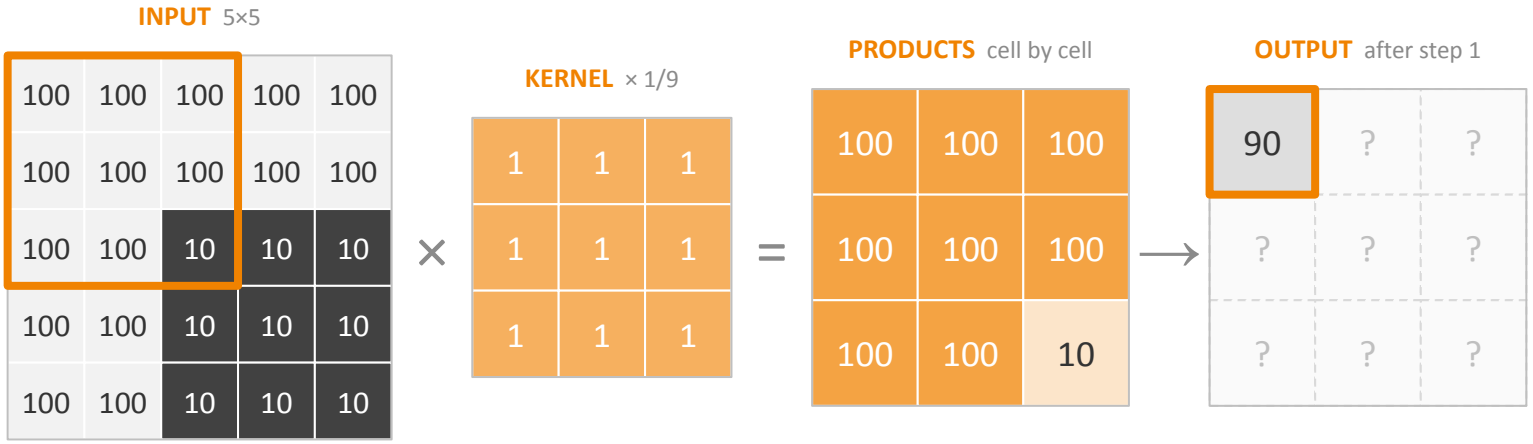
r\c	0	1	2	3	4
0	100	100	100	100	100
1	100	100	100	100	100
2	100	100	10	10	10
3	100	100	10	10	10
4	100	100	10	10	10

orange = window at (0, 0)

- **100** = bright skin, **10** = dark lesion: the corner of a lesion border, like the MILK10k images.
- It contains a **vertical edge** (between columns 1 and 2) and a **horizontal edge** (between rows 1 and 2).
- A 3×3 window fits in **5 − 3 + 1 = 3** positions per direction ⟶ a 3×3 output (no padding).
- We name a window by its **top-left corner** (row, col).

Next: the same 25 numbers through box blur, Gaussian, sharpen, outline, emboss and Sobel. Same input, different kernel, different question.

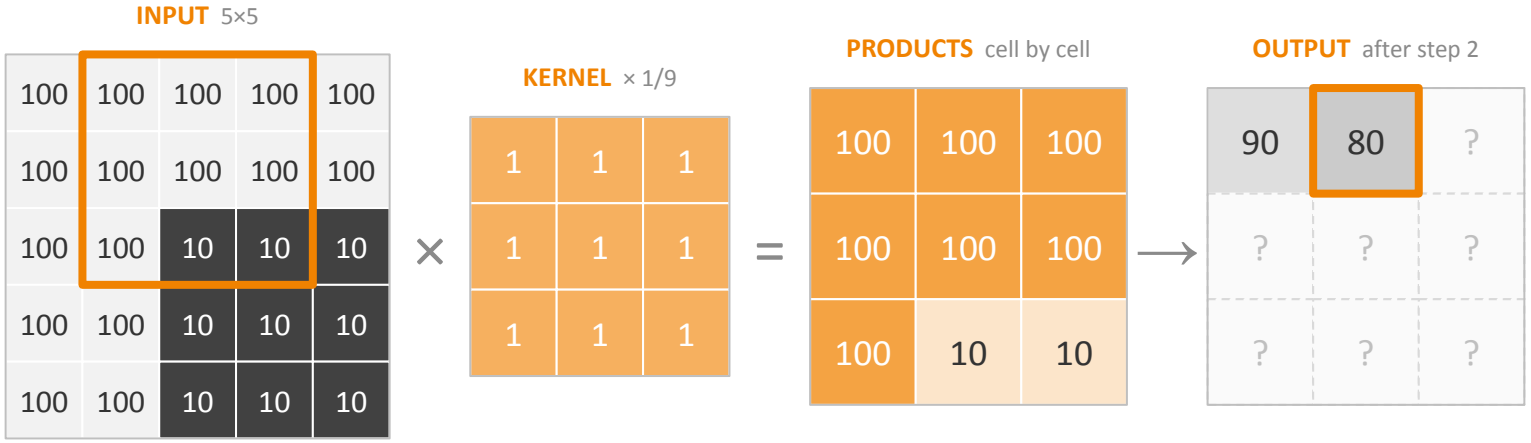
Box Blur, Step 1 of 9: Window at (0, 0)



Σ 100 + 100 + 100 + 100 + 100 + 100 + 100 + 100 + 10 → **810 / 9 = 90**

- Only one dark pixel (10) is under the window, so the output stays bright: 90.
- Write 90 into output cell (0, 0), then slide the window one pixel to the right (stride 1).

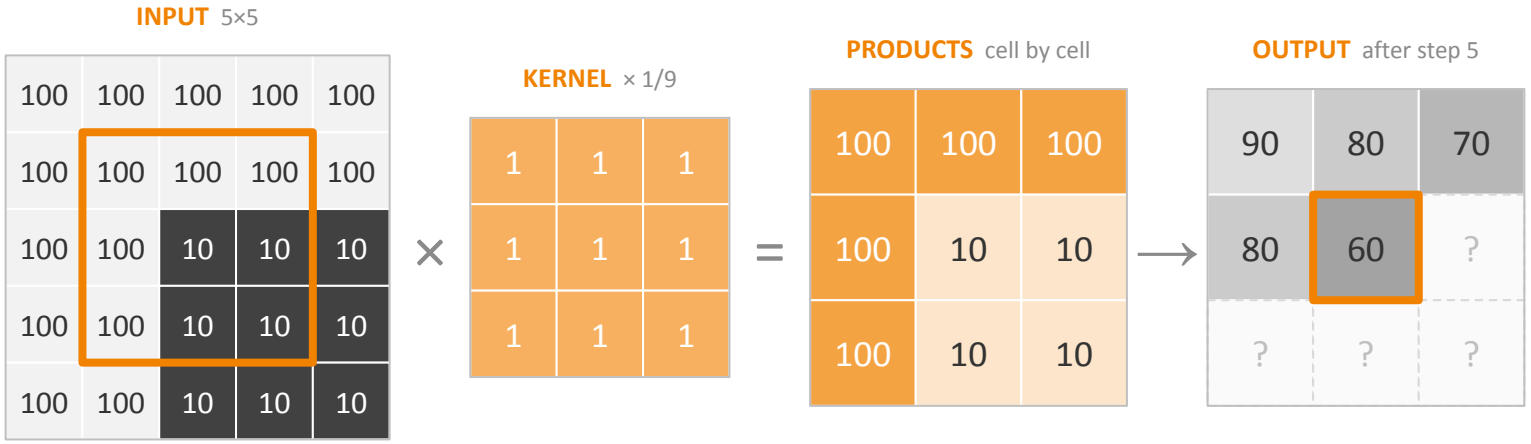
Box Blur, Step 2 of 9: Window at (0, 1)



Σ 100 + 100 + 100 + 100 + 100 + 100 + 100 + 10 + 10 → **720 / 9 = 80**

- Two dark pixels under the window now, so the output drops to 80.
- Same kernel, same arithmetic, new position. That repetition IS the convolution.

Box Blur, Step 5 of 9: Window at (1, 1)



Σ 100 + 100 + 100 + 100 + 10 + 10 + 100 + 10 + 10 → **540 / 9 = 60**

- Steps 3 and 4 work the same way (70, 80). At step 5, four of the nine pixels are dark: 60.
- The output gets darker exactly where the lesion begins, but gradually: the edge becomes a ramp.

Box Blur: All 9 Positions ⟶ The Full Output

INPUT 5×5

100	100	100	100	100
100	100	100	100	100
100	100	10	10	10
100	100	10	10	10
100	100	10	10	10

⟶

OUTPUT 3×3, valid

90	80	70
80	60	40
70	40	10

(0,0)
810 / 9 = **90**

(0,1)
720 / 9 = **80**

(0,2)
630 / 9 = **70**

(1,0)
720 / 9 = **80**

(1,1)
540 / 9 = **60**

(1,2)
360 / 9 = **40**

(2,0)
630 / 9 = **70**

(2,1)
360 / 9 = **40**

(2,2)
90 / 9 = **10**

- The sharp jump 100 ⟶ 10 became a smooth ramp **90** ⟶ **60** ⟶ **10**. That is what blur means, in numbers.
- The output is **3×3, not 5×5**: the window cannot be centred on border pixels. Padding fixes this (next).

Padding and Stride

Two extra choices control the shape of the output feature map:

Parameter	What it controls	Common choice
Padding	Whether to pad the border so the kernel can be centered on edge pixels	'same' (pad to keep output size = input size) or 'valid' (no padding, output shrinks)
Stride	How many pixels the kernel moves between applications	1 (dense) — or 2+ to downsample while filtering

Output size formula:

$$H_{out} = \left\lfloor \frac{H_{in} + 2P_H - K_H}{S_H} \right\rfloor + 1$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2P_W - K_W}{S_W} \right\rfloor + 1$$

Padding by Hand: Zeros vs. Mirror

ZERO-PADDED INPUT 7×7

0	0	0	0	0	0	0
0	100	100	100	100	100	0
0	100	100	100	100	100	0
0	100	100	10	10	10	0
0	100	100	10	10	10	0
0	100	100	10	10	10	0
0	0	0	0	0	0	0

→

OUTPUT, ZERO PAD 5×5

44.4	66.7	66.7	66.7	44.4
66.7	90	80	70	46.7
66.7	80	60	40	26.7
66.7	70	40	10	6.7
44.4	46.7	26.7	6.7	4.4

OUTPUT, MIRROR PAD 5×5

100	100	100	100	100
100	90	80	70	70
100	80	60	40	40
100	70	40	10	10
100	70	40	10	10

Size check

$(n + 2p - k) / s + 1$

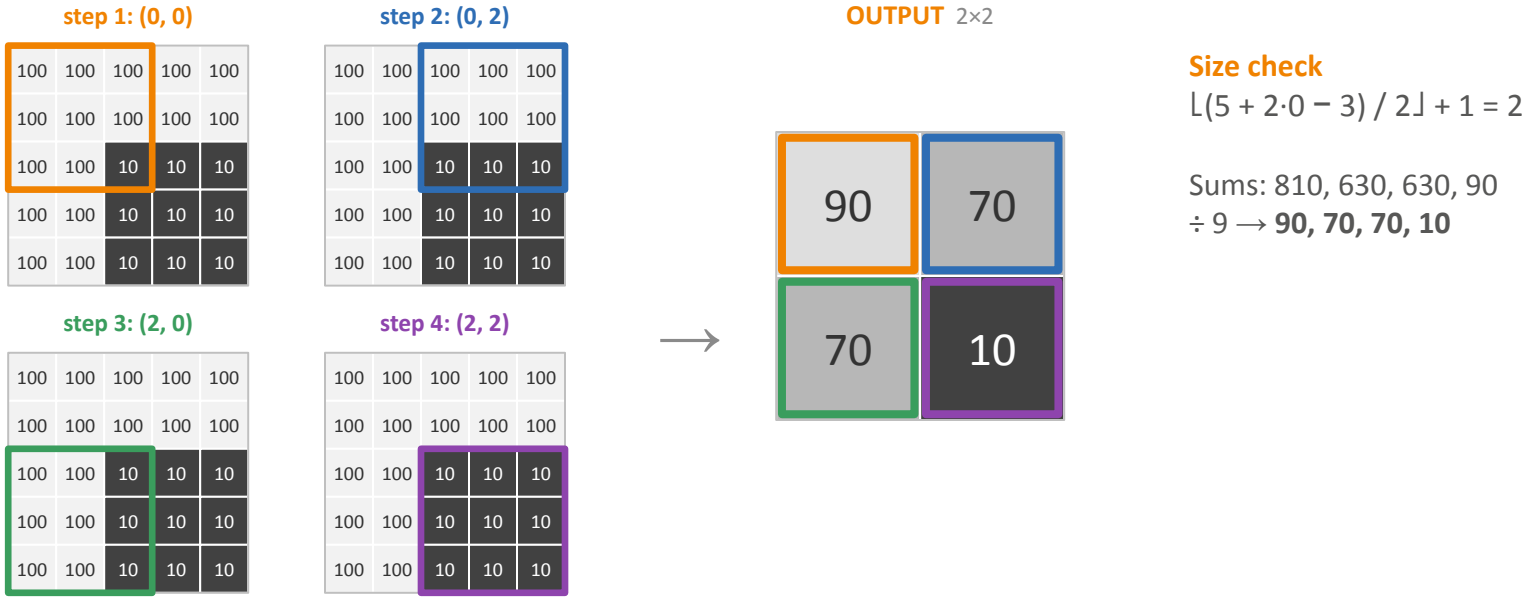
$= (5 + 2 \cdot 1 - 3) / 1 + 1$

= 5 ✓ same size

Corner, zero pad: (0+0+0 + 0+100+100 + 0+100+100) / 9 = 400 / 9 = **44.4** **Mirror pad:** 900 / 9 = **100**

- Zero padding invents black pixels, so the output gets a dark frame (44.4 instead of 100): fake edges at the border.
- Mirror padding repeats the border pixels instead, so no fake edges.

Stride by Hand: Jump 2 Pixels at a Time



Size check

$\lfloor (5 + 2 \cdot 0 - 3) / 2 \rfloor + 1 = 2$

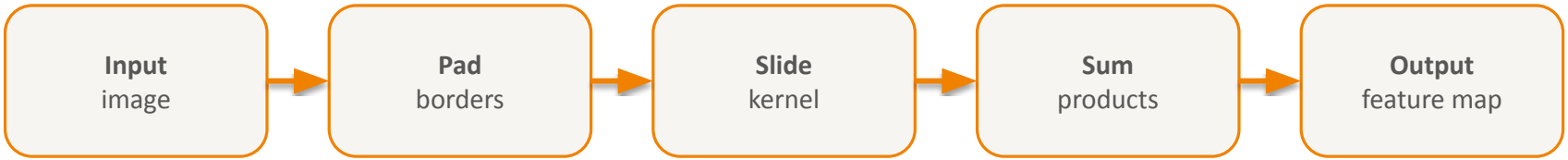
Sums: 810, 630, 630, 90
÷ 9 ⟶ **90, 70, 70, 10**

- Same kernel and arithmetic; we just skip every other position. 4 windows instead of 9.
- The result is the four corners of the stride-1 output: half the resolution. CNNs use stride 2 to downsample.

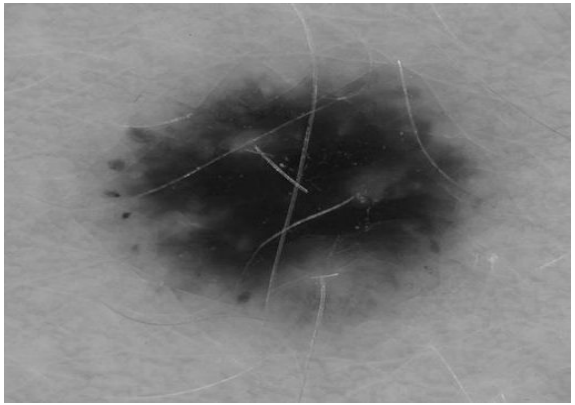
The Filtering Pipeline

? *Blur, sharpen, and edge detection look totally different. Do they share the same underlying steps?*

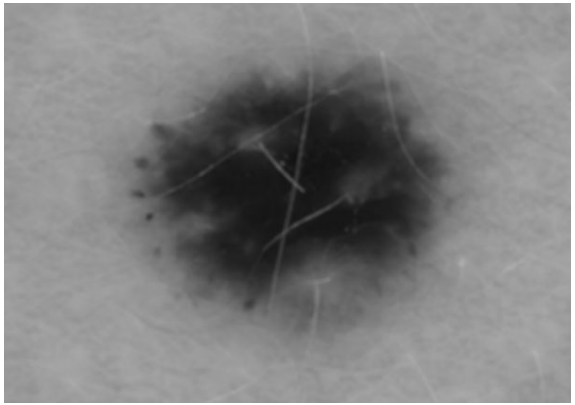
Every classical filter — and every CNN layer — follows this shape:



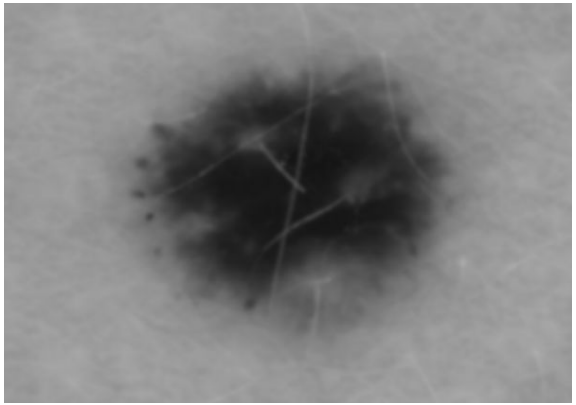
Blur: Box vs. Gaussian



original (grayscale)



box blur, 5×5



Gaussian blur, σ=2.2

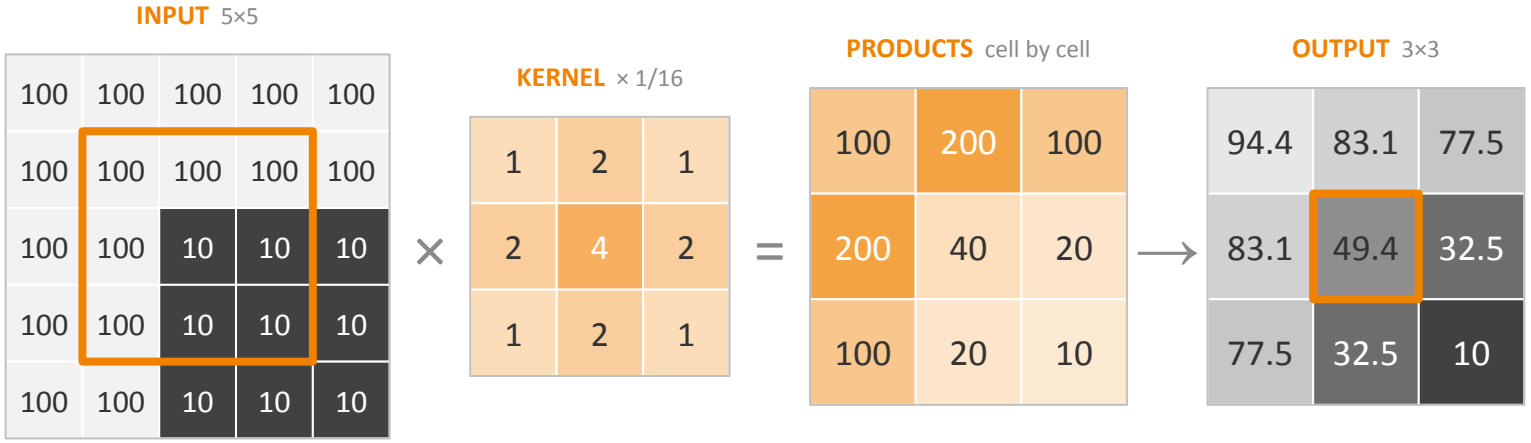
Why Average? Box vs. Gaussian Blur

? *Both blur by averaging neighbors -- so why do the two actually look different?*

Both blur by averaging neighbors — they differ in *how much weight* each neighbor gets:

Kernel	Weighting	Visual effect
Box blur	Every neighbor weighted equally ($1/n^2$)	Fast, but can look 'blocky' — sharp-edged blur
Gaussian blur	Weight falls off smoothly with distance from center (bell curve)	Smoother, more natural-looking blur; the de facto standard

Gaussian Blur by Hand: Weighted Average



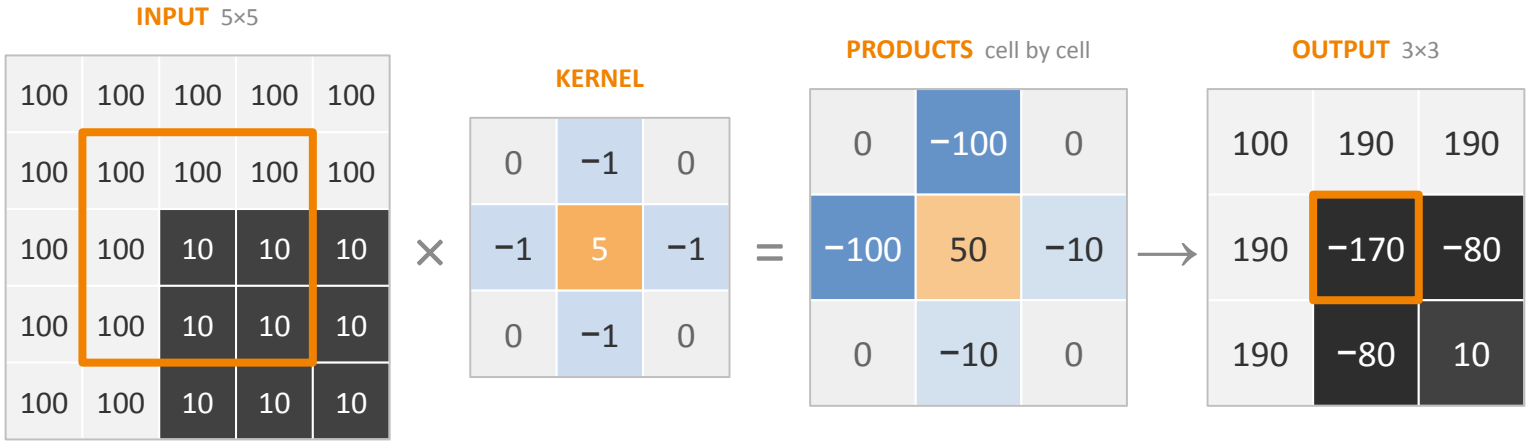
Σ 100 + 200 + 100 + 200 + 40 + 20 + 100 + 20 + 10 → **790 / 16 = 49.4**

- Same window as box-blur step 5, but the centre weighs 4/16 = 25% (box blur: 1/9 ≈ 11%), corners only 1/16.
- So the dark pixels near the centre count more: 49.4 instead of 60. Smoother, yet edges stay more faithful.

Sharpening — Amplifying Differences

- A sharpen kernel does the *opposite* of blur: it **boosts** the center pixel relative to its neighbors.
- A classic sharpen kernel: `[[0,-1,0], [-1,5,-1], [0,-1,0]]`
- Notice the weights sum to **1** — this preserves overall brightness (a flat, uniform region stays unchanged), while regions with local contrast get amplified.
- Conceptually: sharpen = original + (original − blurred version) — it adds back an exaggerated version of the local detail that blur would remove.

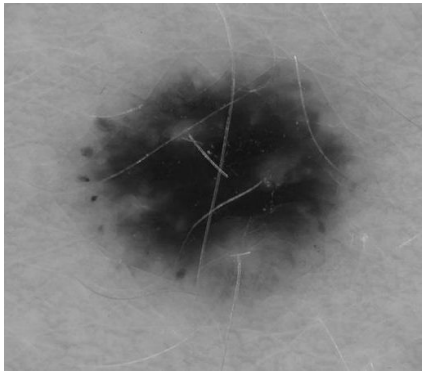
Sharpen by Hand: Overshoot on Both Sides



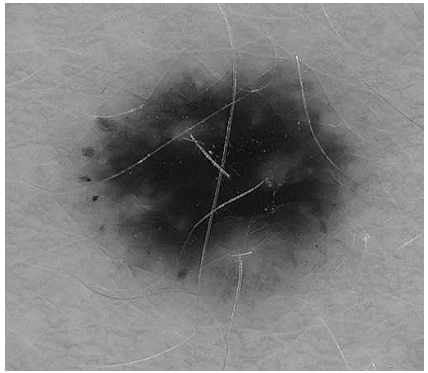
Σ −100 − 100 + 50 − 10 − 10 (zeros skipped) = **−170**

- Weights sum to 1, so flat skin stays 100 (top-left). At the edge: bright side → 190, dark side → −170.
- Bright gets brighter, dark gets darker: that 'halo' is why edges pop. Real images must clip to [0, 255].

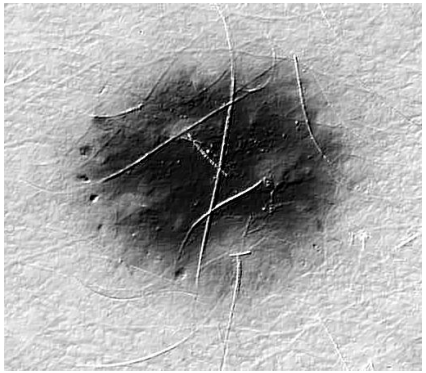
Sharpen, Emboss, Outline



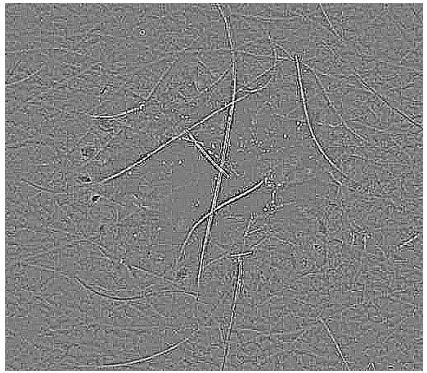
original



sharpen



emboss

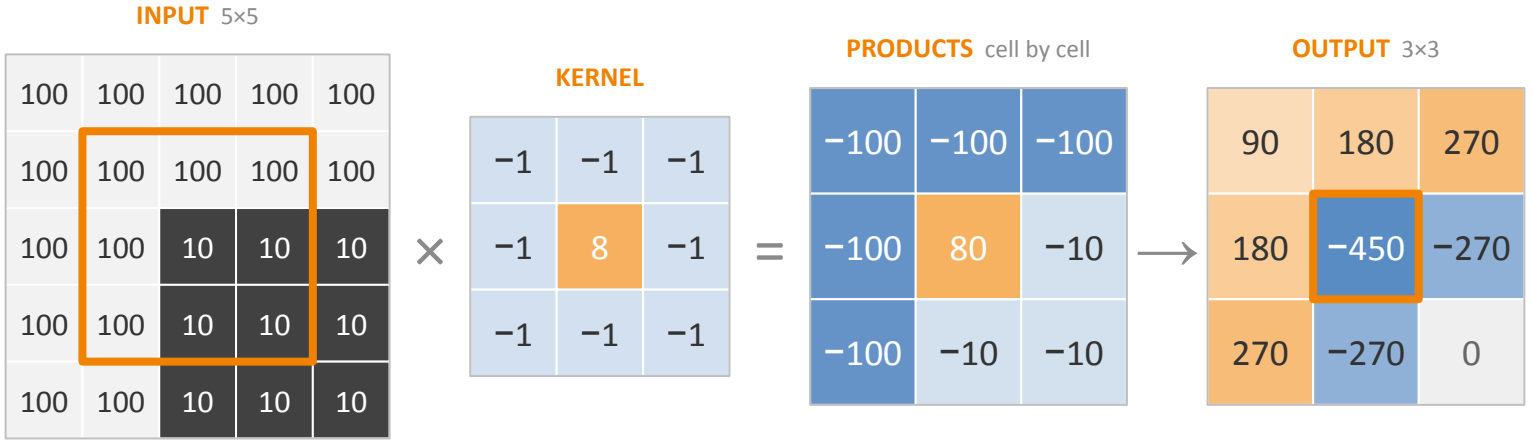


outline

- **Sharpen**: Enhances edges and fine details, making the image appear clearer and more defined.
- **Emboss**: Enhances edges with contrasting light and dark areas, creating a raised, 3D relief effect.
- **Outline**: Detects and emphasizes boundaries between regions, producing an image containing mainly the contours/edges of objects.

** Outline and emboss can produce extreme pixel values. To visualize them properly, we ignore the most extreme 1% at each end and stretch the remaining 98% of values across the black-to-white range.*

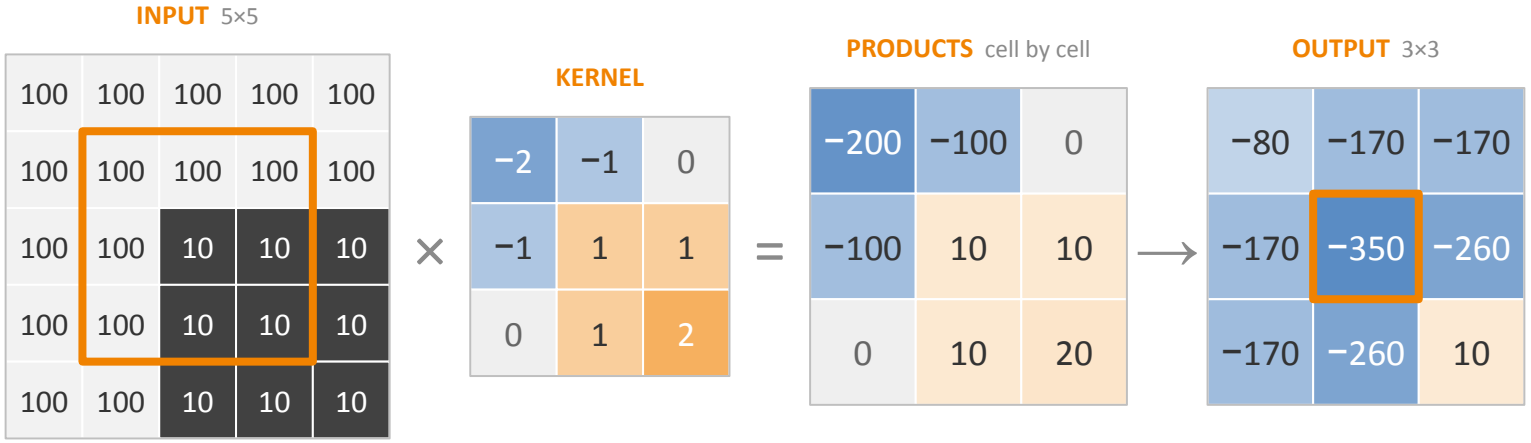
Outline (Edge) Kernel by Hand



Σ −100 − 100 − 100 − 100 + 80 − 10 − 100 − 10 − 10 = **−450**

- Weights sum to 0, so flat regions give exactly 0 (bottom-right, inside the lesion).
- Large values only around the border, in every direction: an all-direction edge detector. Sign = which side is brighter.

Emboss by Hand: A Diagonal Detector



Σ −200 − 100 − 100 + 10 + 10 + 10 + 20 (zeros skipped) = **−350**

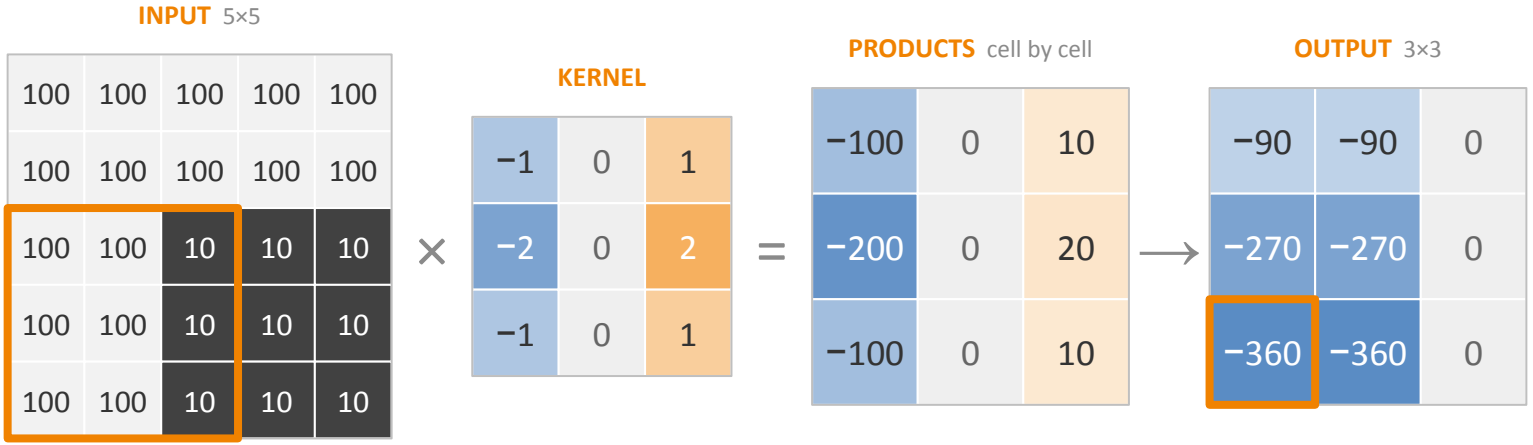
- Negative weights top-left, positive bottom-right: it measures change along the ↘ diagonal (sum = 1).
- Bright ⟶ dark along that diagonal gives a strong negative value: the 3-D 'relief' look. Usually +128 for display.

Edge Detection — Finding Rapid Change

? *How would you teach a computer to notice exactly where a lesion's border is?*

- An **edge** is a place where pixel intensity changes rapidly — the mathematical idea of a large **gradient**.
- The **Sobel operator** estimates the gradient in each direction using two kernels:
- **Gx** (horizontal gradient) = `[[-1,0,1],[-2,0,2],[-1,0,1]]` — **Gy** (vertical gradient) = `[[-1,-2,-1],[0,0,0],[1,2,1]]`
- Combine them as **magnitude = $\sqrt{(Gx^2 + Gy^2)}$** to get overall edge strength, regardless of direction.

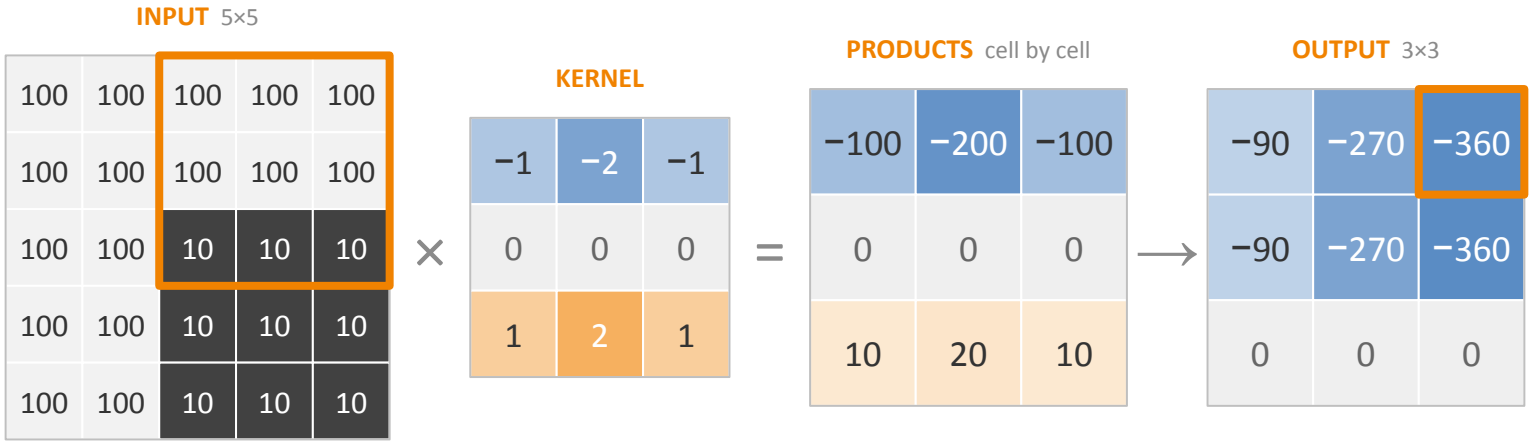
Sobel Gx by Hand: Vertical Edges



Σ −100 + 10 − 200 + 20 − 100 + 10 (zeros skipped) = **−360**

- Left columns bright (100), right column dark (10), so −360: intensity drops left → right. A vertical edge.
- Last output column = 0: there the edge is horizontal, and Gx only sees change from left to right.

Sobel Gy by Hand: Horizontal Edges



Σ −100 − 200 − 100 + 10 + 20 + 10 (zeros skipped) = **−360**

- Top rows bright, bottom row dark, so −360: intensity drops top → bottom. A horizontal edge.
- Last output row = 0: no vertical change there. Gx and Gy each answer one direction.

Edge Strength: Combining Gx and Gy

Gx vertical edges

−90	−90	0
−270	−270	0
−360	−360	0

&

Gy horizontal edges

−90	−270	−360
−90	−270	−360
0	0	0

→

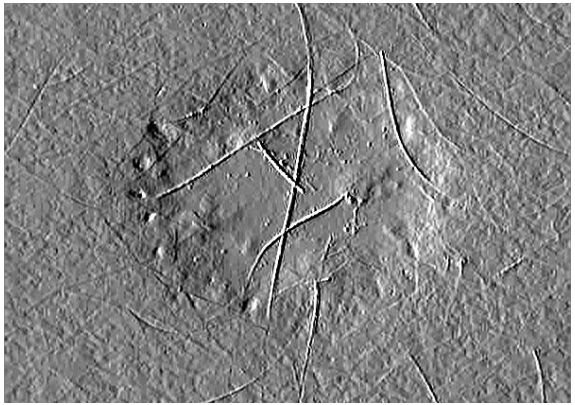
MAGNITUDE $\sqrt{(Gx^2 + Gy^2)}$

127.3	284.6	360
284.6	381.8	360
360	360	0

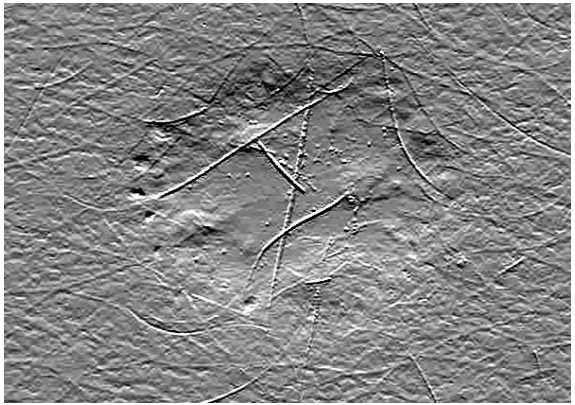
Corner (1, 1): $\sqrt{((-270)^2 + (-270)^2)} = \sqrt{145\,800} = \mathbf{381.8}$ **(0, 2):** $\sqrt{(0^2 + (-360)^2)} = \mathbf{360}$

- Gx and Gy each see one direction; the magnitude sees edges in any direction: the whole L-shaped border lights up.
- Inside the lesion, and on flat skin far from it, the magnitude is 0. This is the 'edge map' you saw on the real lesion.

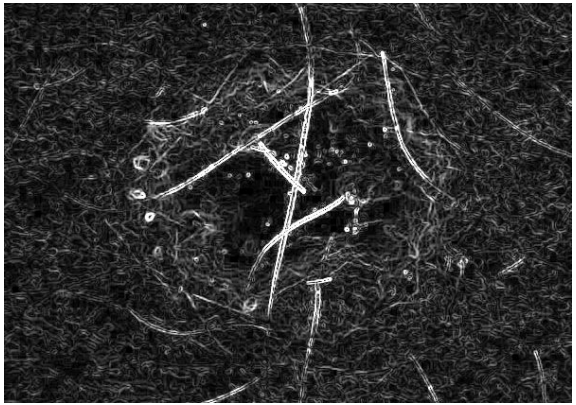
Sobel Gradients on a Real Lesion



Gx — horizontal gradient



Gy — vertical gradient



magnitude $\sqrt{(Gx^2+Gy^2)}$

Your Turn: 4 Kernels, 1 Window

INPUT window at (1, 2)

100	100	100	100	100
100	100	100	100	100
100	100	10	10	10
100	100	10	10	10
100	100	10	10	10

BOX BLUR × 1/9

1	1	1
1	1	1
1	1	1

= ?

SHARPEN

0	−1	0
−1	5	−1
0	−1	0

= ?

SOBEL GX

−1	0	1
−2	0	2
−1	0	1

= ?

SOBEL GY

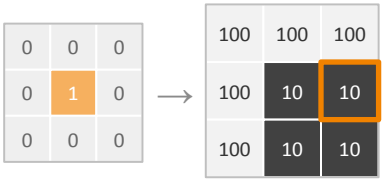
−1	−2	−1
0	0	0
1	2	1

= ?

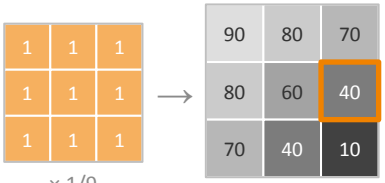
Hint: one of the four answers is exactly 0. Can you tell which before computing anything? Check your answers on the next slide (cell (1, 2) of each output).

One Input, Eight Kernels, Eight Answers

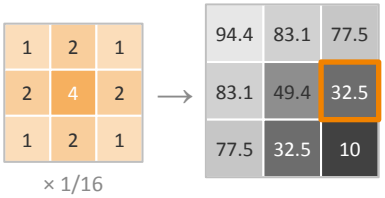
IDENTITY sum = 1



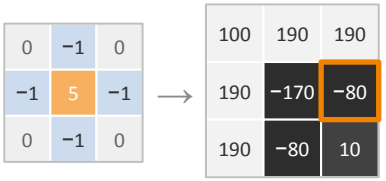
BOX BLUR sum = 1



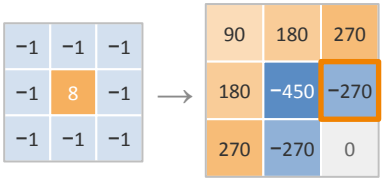
GAUSSIAN sum = 1



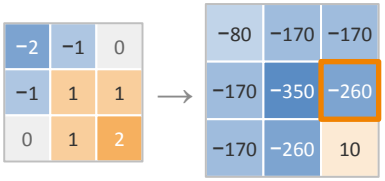
SHARPEN sum = 1



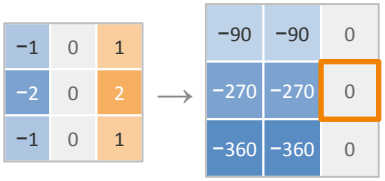
OUTLINE sum = 0



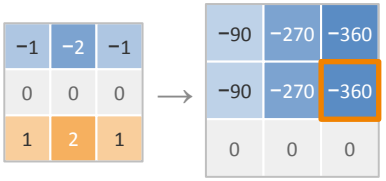
EMBOSS sum = 1



SOBEL GX sum = 0



SOBEL GY sum = 0



- **Weights sum to 1** (identity, blurs, sharpen, emboss): flat skin keeps its value, so brightness is preserved.
- **Weights sum to 0** (outline, Sobel): flat regions become 0 and only edges survive. These are edge detectors.

Kernels as Feature Detectors

? *So far WE chose every kernel's weights by hand. What if a model could learn them instead?*

- Every kernel we've seen **asks a specific question** of each neighborhood: 'is this smooth?' (blur), 'is there a sharp edge here, and in which direction?' (Sobel), 'is the center a local peak?' (sharpen).
- This reframing is the conceptual bridge to deep learning: a kernel is not just an image-processing trick — it is a **feature detector**.
- Different kernels detect different features. A bank of many kernels, applied in parallel, gives many feature maps — each answering a different local question about the image.

KEY IDEA

So far, **we** chose the kernel weights by hand. What if a model could *learn* which weights are most useful for a task? That is exactly what a CNN's convolutional layer does.

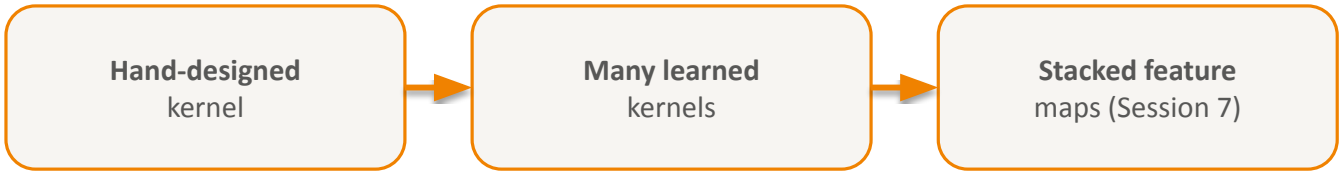
A Technical Note — Convolution vs. Correlation

- True mathematical convolution flips the kernel 180° before sliding it; what we (and virtually every deep learning framework) actually compute is **cross-correlation** — no flip.
- For a learned kernel, this distinction is irrelevant: the network simply learns whichever orientation is useful.
- `PyTorch`'s `nn.Conv2d`, like most frameworks, implements cross-correlation but calls it "convolution".
- What we've been calling convolution is technically cross-correlation, because we don't flip the kernel. This is the convention used in deep learning frameworks such as PyTorch and TensorFlow.

From One Filter to a Convolutional Layer

? *A CNN layer is 'just many kernels' -- so what's actually different from doing it by hand?*

A CNN layer is simply many kernels, applied in parallel, and — critically — learned:



Part 1 Recap — Key Takeaways

- A filter combines each pixel with using a fixed set of

Part 1 Recap — Key Takeaways

- A filter combines each pixel with its **local neighborhood** using a fixed set of weights (the kernel).
- Convolution = kernel weights by the pixels underneath, then — repeated at every position. them.

Part 1 Recap — Key Takeaways

- A filter combines each pixel with its **local neighborhood** using a fixed set of weights (the kernel).
- Convolution = **multiply** kernel weights by the pixels underneath, then **sum** — repeated at every position.
- and control the output size; the same formula reappears when sizing CNN layers.

Part 1 Recap — Key Takeaways

- A filter combines each pixel with its **local neighborhood** using a fixed set of weights (the kernel).
- Convolution = **multiply** kernel weights by the pixels underneath, then **sum** — repeated at every position.
- Padding and stride control the output size; the same formula reappears when sizing CNN layers.
- averages (smooths), amplifies local contrast, estimates gradients (finds edges).

Part 1 Recap — Key Takeaways

- A filter combines each pixel with its **local neighborhood** using a fixed set of weights (the kernel).
- Convolution = **multiply** kernel weights by the pixels underneath, then **sum** — repeated at every position.
- Padding and stride control the output size; the same formula reappears when sizing CNN layers.
- Blur averages (smooths), sharpen amplifies local contrast, Sobel estimates gradients (finds edges).
- A kernel is a detector — deep learning's core idea is to **learn** these weights instead of hand-designing them.

Part 1 Recap — Key Takeaways

- A filter combines each pixel with its **local neighborhood** using a fixed set of weights (the kernel).
- Convolution = **multiply** kernel weights by the pixels underneath, then **sum** — repeated at every position.
- Padding and stride control the output size; the same formula reappears when sizing CNN layers.
- Blur averages (smooths), sharpen amplifies local contrast, Sobel estimates gradients (finds edges).
- A kernel is a feature detector — deep learning's core idea is to **learn** these weights instead of hand-designing them.

Part 1 Recap — Key Takeaways

- Before the break:
- Output size ?
  - 32x32 input image
  - 5x5 kernel
  - Padding of 2
  - Stride of 1

TIME FOR A BREAK

20 Minutes

Stretch, grab a coffee, and be back on time.

eada
business school
barcelona

SESSION 4

Part 2 — Hands-on Python

Implementing convolution from scratch

eada
business school
barcelona

Practice Objectives

- Create a Python Notebook and follow the code.

Wrap-Up — What We Built Today

- Convolution worked by hand on a 5×5 image, then a from-scratch conv2d implementation — you now know exactly what a convolution is, not just how to call one.
- Blur, sharpen and Sobel edge-detection applied to a real MILK10k image, both manually and with scipy.ndimage.correlate
- A measured, order-of-magnitude comparison between naive and vectorized convolution.

Homework — Before Session 5

- **Explore learned filters:** Apply a set of filters to different MILK10k images from different diagnosis classes. Do edges or textures look visually different between malignant and benign lesions?
- **Gaussian blur:** Read the scipy.ndimage.gaussian_filter documentation and identify what the sigma parameter controls. Experiment with different values of sigma and observe the effect.
- **Canny edge detection:** Read about the Canny edge detector and **implement it in Python as a function**. Apply your function to one MILK10k image and compare the result with a Sobel edge detector. Briefly describe the main differences you observe.

Coming Up Next & References

- **Next — Session 5: Classical Machine Learning for Vision.** We use exactly the filters from today to extract features — color histograms, texture, edge statistics — and feed them into classical ML models as a project baseline.

References & further reading

- MIT Visionbook — Filtering: `visionbook.mit.edu`